

Optimized Parallelization Framework for Real-Time Seismic Event Detection in Computationally-Constrained Networks

Constantinos G. Skevofilax¹, Victor Salles¹, Camilo Eduardo Muñoz Lopez¹, Daniel Siervo¹, Omar M. Saad², Yangkang Chen¹, and Alexandros Savvaidis¹

Abstract

This study introduces **EQCCTPro**, an optimized parallel processing framework that significantly improves the computational efficiency of deploying deep learning-based seismic phase-picking models—such as the Earthquake Compact Convolutional Transformer (EQCCT)—for analyzing seismic activity across entire seismic networks. Our findings demonstrate that our optimized parallelization strategy enables DL-based detection models to continuously analyze and pick seismic waveforms from large-scale seismic networks in real time—a capability that was previously difficult to achieve due to limitations in hardware availability, computational efficiency, and resource optimization. By implementing our optimized parallelization strategy, we were able to analyze one minute of waveform data from 99 unique seismic stations using only 15 CPUs in under 30 seconds. Using **EQCCTPro**, seismic networks worldwide can seamlessly integrate DL models, like EQCCT, into their event detection systems, enabling faster, more accurate, and more efficient seismic event detection through continuous real-time processing, all without the need for intensive computational resources.

Cite this article as First A., A. Second, and A. Third (2022). Optimized Parallelization Framework for Real-Time Seismic Event Detection in Computationally-Constrained Networks, *The Seismic Record* 0(0), 1–11, doi: [00.0000/0000000000](https://doi.org/10.0000/0000000000).

[Supplemental Material](#)

Introduction

In seismology, having precise P- and S-wave arrival times is essential for building accurate earthquake catalogs. However, achieving this precision is challenging, as manually picking arrivals from large datasets is both time-intensive and inefficient. As a result, the seismology community has a pressing need for accurate and reliable automated picking methods that enable real-time seismicity monitoring. These methods are critical for enabling swift and effective emergency responses to seismic events, including evacuations, damage assessments, and public warnings, especially in regions with high seismic activity.

Traditionally, the short-term and long-term average (STA/LTA) algorithm has been widely adopted for automating earthquake detection and phase picking. This approach identifies events by comparing the ratio of short-term and long-term averages of waveform amplitudes against a pre-defined threshold in real-time (Saad et al. (2023), Yu et al. (2024), Chen et al. (2024)). While STA/LTA provides a degree of event detection automation, its reliance on a fixed ratio threshold to detect an event makes it highly sensitive to background noise, leading to an increased likelihood of false positives and negatives. Extensive tuning is required to accommodate varying noise levels across different stations, increasing the time and labor needed for effective implementation.

To address these limitations, deep learning (DL) has emerged over the past decade as a powerful alternative, offering solutions to many of these challenges (Zhu and Beroza (2018); Mousavi et al. (2020); Mousavi and

*Corresponding author: auname@abc.org

© 2025. The Authors. This is an open access article distributed under the terms of the CC-BY license, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Beroza (2022); Saad and Chen (2022); Saad et al. (2021); Feng et al. (2022); Saad et al. (2022,?). DL methods have shown superior detection accuracy over traditional detection approaches in terms of precision and recall (Zhu and Beroza (2018); Mousavi et al. (2020); Xiao et al. (2021); Tan et al. (2021)) while also being generalizable across various seismic networks (Münchmeyer et al. (2022)). This combination of accuracy and adaptability ensures that the picked events referenced by emergency services, government authorities, and industry leaders are more reliable and readily available, enabling quicker responses to seismic hazards and supporting informed decision-making. Despite these advancements, many of the most advanced DL-based pickers have struggled to generalize to station networks they were not previously trained on, remain highly sensitive to varying extremes in noise and signal conditions (Mousavi et al. (2020); Münchmeyer et al. (2022)). To address these challenges, the Earthquake Compact Convolutional Transformer (EQCCT), a DL-ML model, was developed (Saad et al. (2023)). EQCCT has shown superior performance, generalizability and precision in detecting P and S wave arrivals, outperforming popular DL-based methods such as EQTransformer and PhaseNet, making it a highly reliable option for automating phase detection.

Building on this innovation, the Texas Seismological Network and Seismology Research Team (TexNet, Savvaidis et al. (2019)), has incorporated EQCCT into its production waveform analysis workflow. This incorporation enables more efficient analysis of seismic waveforms collected from over 300 active stations across the network. By leveraging EQCCT's rapid detection capabilities, TexNet accelerates waveform analysis while allowing human analysts to verify highly accurate ML-generated picks—enhancing the overall reliability and responsiveness of seismic event detection.

Expanding on this integration, TexNet aims to leverage EQCCT to develop an **advanced, real-time, autonomous, DL-based, earthquake detection system**. The proposed detection system must analyze multiple waveforms from a large seismic network in **under one minute** while operating with **limited computational resources** (e.g., a maximum of 15 CPUs). This presents a significant challenge, as the system must effectively utilize the available computational hardware to meet the strict real-time processing requirements.

In this paper, we present a highly efficient parallelized computing framework that enables EQCCT—and poten-

tially other DL-based pickers—to analyze up to 99 unique waveforms from a large seismic network in under 30 seconds, even while under constrained computing conditions. By parallelizing the sequential processing approach of *EQCCTOne*, our method supports real-time seismicity monitoring across diverse networks. It combines EQCCT's strong generalizability with our framework's efficient use of available hardware, achieving fast, scalable, and accurate seismic event detection.

Methodology

In its current production implementation at TexNet, EQCCT operates in "postevent mode", analyzing previously recorded waveforms and generating picks that human analysts can review. The method that is responsible for collecting historical waveforms from seismic stations and applying EQCCT to analyze them is known as *EQCCTOne* (Figure 1). The *EQCCTOne* method operates as follows: **1. Resource Allocation:** The user defines the number of computing resources (i.e., CPUs) available for the method and provides station codes for waveform data retrieval. **2. Timeframe Definition and Timechunk Division:** The user defines a **total timeframe window**, representing the overall period of interest to be analyzed (e.g., 1 day). This timeframe is divided into smaller "**timechunks**" of user-specified durations (e.g., 1 minute, 5 minutes, 30 minutes), with the total number of timechunks calculated as:

$$\text{Number of Timechunks} = \frac{\text{Total Timeframe Window}}{\text{Timechunk Time Length}}$$

For instance, in Figure 1, if the total timeframe window is three minutes and the timechunk length is one minute, each station will return three timechunks, each one minute long. With 99 stations, this results in 297 unique waveforms. **3. Waveform Compilation:** These waveforms are compiled into a list, with each unique one-minute timechunk corresponding to a specific station as the source. **4. Sequential Processing:** The waveform list is processed **sequentially**, one at a time, with EQCCT (i.e., the ML model) analyzing each waveform and generating a pick if a seismic event is detected. *EQCCTOne* then saves the resulting pick(s) to file. The process repeats sequentially until all waveforms are processed.

To evaluate how well *EQCCTOne* would perform under real-time processing conditions, we applied the method to

a controlled trial simulating waveform processing across a large-scale seismic network.

To simulate a real-world deployment, 99 unique one-minute waveforms were sourced from 99 TexNet seismic stations. The one-minute timeframe was chosen as it represents both the minimum waveform length required for EQCCT to function (Saad et al. (2023)) and the designated input waveform length for TexNet's real-time deployment. Since further waveform segmentation is unnecessary at this input length, the one-minute timeframe remained unchanged and was not divided into smaller timechunks.

To assess *EQCCTOne*'s scalability, the number of sequentially processed stations was systematically increased. Testing began with a single station and increased in increments of **one** until reaching **10**, after which the step size increased to **five** until the full computational workload of 99 stations (or 99 one-minute waveforms) was reached.

For *EQCCTOne* to be viable for real-time deployment, it must analyze all unique one-minute waveforms and generate picks originating from the same one-minute timeframe in under one minute, ideally ≤ 30 seconds. Exceeding this threshold compromises real-time functionality—as if *EQCCTOne* takes longer than one minute process all waveforms, incoming waveforms from the next time window must wait until the previous set is fully processed, shifting the system from a real-time to near-real-time operation. The 30 second constraints gives *EQCCTOne* a 30 second buffer window for any issues with the processing of waveforms.

In addition to the above constraint, *EQCCTOne* was evaluated to determine whether it could meet runtime requirements while operating with limited computational resources—an essential consideration for real-world deployment. Many seismic networks may consider integrating DL-based pickers like EQCCT into their automated event detection systems, but concerns about sufficient resource availability may cause hesitation. Available computational hardware may be insufficient for active deployment, or lack the efficiency needed to process the required network size in real-time, or be improperly configured for optimal utilization of DL-based picking methods.

To address these concerns, the real-world scenario test described above were conducted under varying CPU limits. The test was performed on a **128-CPU system** equipped with **AMD Ryzen Threadripper PRO 7985WX processors**, each featuring 64 cores. To systematically evaluate per-

formance, computational usage was incrementally reduced from a maximum of 15 CPUs down to a single CPU.

To ensure strict control over the allocated computational resources and prevent unintended access to additional cores, we employed the *sched_setaffinity* function—a Linux utility that binds processes to specific CPU cores. This enforcement ensured that each scenario trial had exclusive access only to its assigned cores, effectively simulating a constrained computing environment. For instance, in the **15-CPU trial**, only cores **0 through 14** were assigned to that specific *EQCCTOne* process. This controlled allocation was consistently applied across all 15 trials (Figure 2), ensuring that each instance operated within its intended constraints. By implementing these measures, the tests accurately reflected *EQCCTOne*'s performance under real-world deployment constraints.

The method's performance was evaluated by measuring the runtime of the picking process. Waveform download time was excluded, focusing only on the duration from organizing all waveforms into the required data structure for EQCCT input to the completion of processing, which includes pick generation and saving.

This methodology provided us a comprehensive assessment of *EQCCTOne*'s simulated real-time deployment performance when analyzing a large-scale, real-world, seismic network under varying computational constraints.

Performance Deficiencies in *EQCCTOne* and the Case for Parallelization

Findings from Table 1 and Figure 4a highlight a significant performance limitation when comparing the EQCCT model's raw processing capability to its use with the *EQCCTOne* methodology. While EQCCT can individually process a single seismic waveform and generate a pick in approximately ~0.15 seconds, integrating the model into *EQCCTOne*'s sequential pipeline leads to dramatically increased runtimes—averaging 251.6 seconds—when processing multiple waveforms across a seismic network. This inefficiency stems from *EQCCTOne*'s design, which processes waveforms in a strictly serial manner (Step 4, Figure 1)—each waveform must wait for the previous one to complete before being processed. As a result, even though each individual task is fast, the cumulative runtime scales poorly with the increasingly number of waveforms in queue.

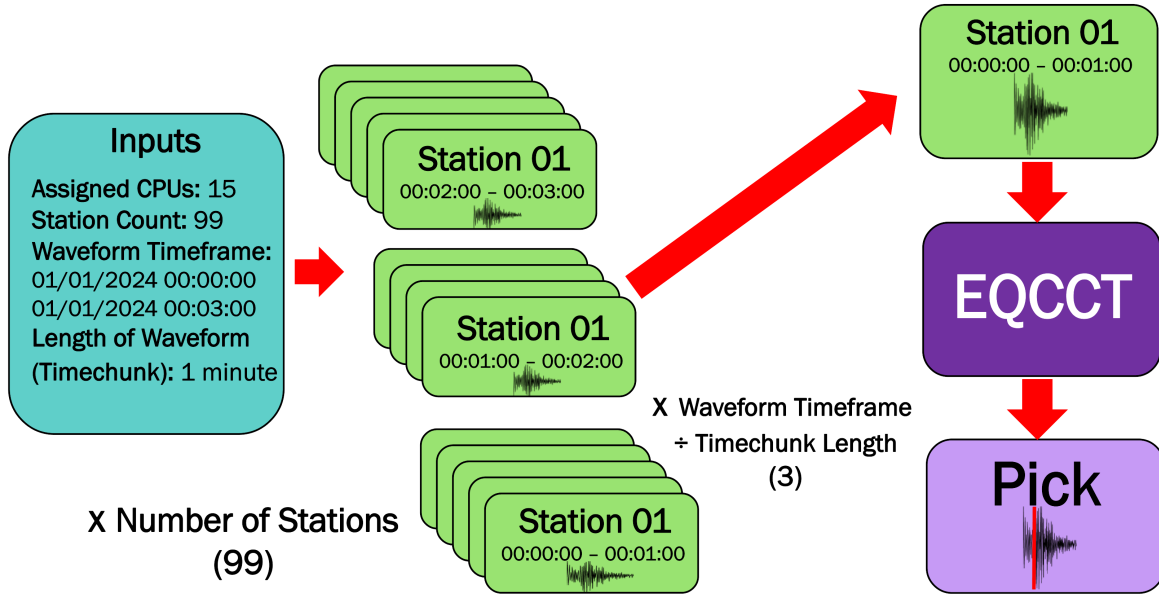


Figure 1. Process Workflow of EQCCTOne in a Sample Test Scenario: A user assigns **15 CPUs** to EQCCTOne, selects **99 stations** as the reference data source, and specifies a **three-minute total waveform timeframe** with a **timechunk length of one minute**. This configuration divides the waveforms into **three timechunks per**

station, resulting in **297 unique waveforms** to be processed. Each waveform is processed **sequentially** by EQCCT, leading to high runtimes. In this scenario, analyzing and generating picks for all 297 waveforms takes **518.72 seconds** (nearly **nine minutes**).

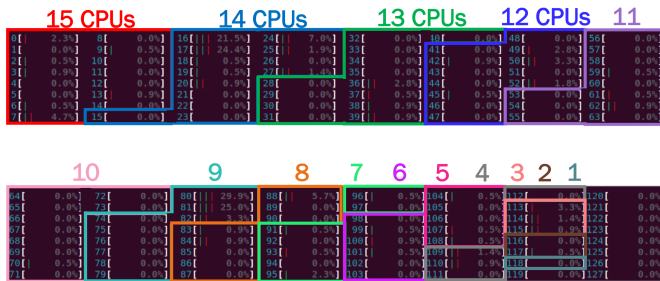


Figure 2. Illustration of the 128-CPU Multi-Processor System Used in our Trials: The `sched_setaffinity` Linux function restricted processes from accessing additional resources by binding each process to a specified set of CPUs based on their system processor IDs. This ensured that each trial remained confined to its designated CPU section, effectively simulating a controlled, resource-constrained environment for EQCCTOne.

According to **Amdahl's Law** (Amdahl (1967)), the speedup achievable from parallelizing a task is fundamentally limited by portion of the task that must remain serial. While initial increases in CPU utilization reduce runtime significantly (evident in the asymptotic decay seen in Figures 4a & 5a), this benefit quickly tapers off. Because

EQCCTOne's processing approach is largely serial, increasing the number of CPUs yields limited performance gains. As the serial bottleneck persists, parallel processing efficiency quickly plateaus, leading to diminishing returns despite additional computing resources (Figure 5a). This limitation is clearly reflected in the flattening of speedup at higher CPU counts and highlights why EQCCTOne's architecture is not scalable. This is further emphasized in Table 1, where the highest runtimes are observed when a large number of stations (e.g., 99) are processed using minimal computing power (e.g., 1 CPU), highlighting the inefficiencies of serial execution. Conversely, the shortest runtimes occur when smaller workloads (e.g., 1 station) are allocated maximum computing resources (e.g., 15 CPUs), demonstrating the performance gains achievable through appropriate workload-to-resource ratios.

To overcome these limitations, we developed **EQCCTPro** (Figure 3), a highly parallelized framework that leverages the open-source library **Ray** (Ray.io (nd), Ali (2023)) to enable DL-pickers like EQCCT to process multiple waveforms concurrently across a set of processing units. Our approach combines EQCCT's inherent processing speed

with the scalability of parallel computing. Unlike its predecessor, *EQCCTPro* distributes multiple waveform tasks across available CPU cores and executes them in parallel, effectively eliminating the bottlenecks associated with a purely sequential execution approach.

Implementation of *EQCCTPro* and its Significant Runtime Enhancements in Enabling Real-Time Seismicity Monitoring

EQCCTPro (Figure 3) retains the foundational steps as *EQCCTOne*—resource allocation, waveform time-frame selection, and division into timechunks (Steps 1–3). However, rather than processing all waveforms sequentially, *EQCCTPro* introduces multi-level parallelization.

EQCCTPro first processes a specified number of timechunks concurrently, denoted as $xS.T$, where $S.T$ represents the number of **simultaneous timechunk processes**. This design enables deep learning pickers such as EQCCT to evaluate multiple time windows in parallel, significantly improving throughput. Similarly, *EQCCTPro* introduces a second layer of parallelism **within each timechunk**, enabling multiple waveforms to be processed concurrently rather than sequentially. This inner-level parallelism, denoted as $xS.P$, reflects the number of **simultaneous DL-predictions** executed within each timechunk.

Together, these enhancements enable EQCCT to operate in parallel across both the **timechunk** and **station** levels, effectively transforming what was previously a sequential workflow into an embarrassingly parallel problem (Herlihy and Shavit (2012)). These user-defined parameters ($xS.T$, $xS.P$) enable full utilization of available computing resources, maximizing system performance and scalability for real-time seismic monitoring.

To assess the runtime impact of this new architecture, we applied the same iterative scalability tests used in the *EQCCTOne* trials to *EQCCTPro*. Since real-time deployments typically involves processing a single one-minute waveform timechunk, the station-level parallelization becomes especially beneficial for maximizing throughput.

To identify the optimal parallelism configuration for any given workload, we modified the tests to systematically vary the number of concurrent DL-predictions ($xS.P$) used across

different combinations of station count and CPU availability. For each test case, the number of concurrent predictions was incrementally increased. For example, with 15 CPUs and a workload of 30 stations, $xS.P$ configurations ranged from 1 to 30. This process was repeated across all scenarios—scaling up to 99 stations—to empirically determine the most efficient parallelization strategy for each workload.

Results

Using the *EQCCTPro* implementation with the same trial framework as *EQCCTOne*, we observed a substantial improvement in runtime efficiency for identical workloads (Figure 4a vs. 4b). On average, *EQCCTPro* reduced the processing time by **68.2%** on average compared to its sequential counterpart (Table 1), even without tuning parallelization parameters. In this baseline configuration, *EQCCTPro* was capable of processing a large seismic network (99 stations) in approximately **80 seconds**, a marked improvement over the **251.6 seconds** required by *EQCCTOne* for the same task.

With optimized parallelization configurations, *EQCCTPro* achieved even greater gains. By tuning the number of concurrent DL-predictions ($xS.P$), the average runtime dropped to just **31.38 seconds**, nearly reaching the 30-second threshold required for real-time seismic event detection. In the fully scaled configuration—which utilized all 15 available CPUs resources and the optimally determined 58 simultaneous predictions—*EQCCTPro* processed all 99 unique waveforms from **99 stations** in just **28.12 seconds**. This represents a **87.1% reduction in runtime** compared to the **217.35 seconds** required by *EQCCTOne* to process the same workload. By converting a previously serial bottleneck into an embarrassingly parallel framework (Figure 5b), *EQCCTPro* not only meets the processing demands of large-scale seismic network monitoring but also enables scalable, real-time performance on modest hardware (Figure 6). It allows systems to process substantially larger workloads while sustaining high speedup gains across a wider range of configurations—something not achievable with a serial processing approach (McCool et al. (2012), Gustafson (1988)).

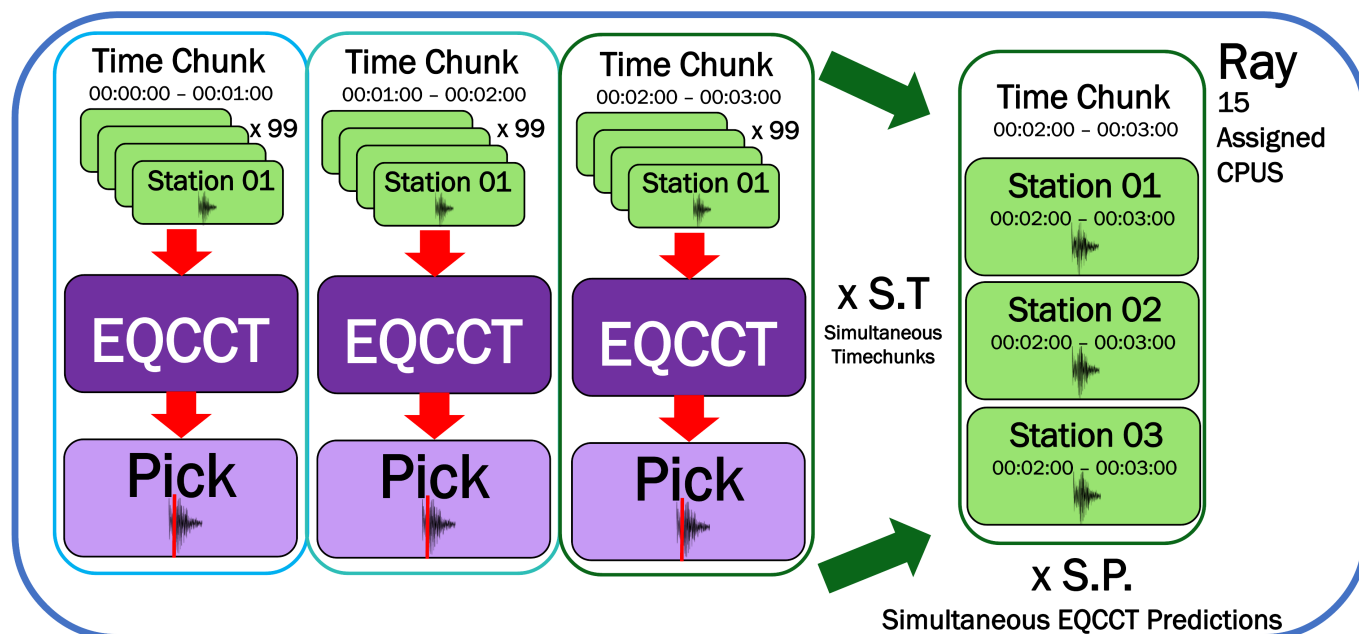


Figure 3. Process Workflow of EQCCTPro under the same test scenario as Figure 1: EQCCTPro leverages Ray to parallelize waveform processing, eliminating the need for sequential execution. In this workflow, three timechunks (minutes 1 through 3), each containing 99 unique waveforms from 99 referenced stations, are processed simultaneously. Ray handles each timechunk independently, distributing workloads across available resources. This parallelization is defined as $xS.T$, the number of simultaneous timechunk processes. Ray further parallelizes EQCCT execution within each timechunk,

enabling $xS.P$ simultaneous EQCCT predictions to run concurrently. By efficiently utilizing allocated resources (e.g., 15 CPUs), multiple waveforms can be processed simultaneously across a timeframe and within each timechunk. As a result, the same workload that previously required 518.72 seconds was reduced to 51.41 seconds—a **90.1% reduction in runtime**. This parallelization approach significantly enhances workflow efficiency, with an average runtime reduction of **68.2%** (Table 1).

Table 1. **Comparative Trial Performance Analysis**

Metric	Min	Max	Avg.
a. Waveform Analysis Time (s)			
EQCCT	0.04	1.78	0.15
b. Method Trial Runtime (s)			
EQCCTOne	29.04	2549.38	251.64
EQCCTPro	4.82	575.26	79.98

Using Gradient Boosting Regressor for Modeling Maximizing EQCCTPro's Performance on Limited Computing Hardware

Building on our findings, we sought to model the relationship between three key variables—CPU count, total station workload, and the number of concurrent predictions—to better understand how they collectively influence runtime performance. Our analysis revealed three distinct trends and interpretable patterns in the performance surface (Figure 7a): **1. An inversely linear decay** between

CPUs and runtime (x-axis vs. z-axis), **2. A non-monotonic, regime-shifting behavior** between concurrent predictions and runtime (y-axis vs. z-axis), and **3. A linear increase** in runtime as a function of total station count (z-axis vs. color gradient). The **inversely linear decay** observed with increasing CPUs is consistent with the expected behavior of embarrassingly parallel workloads (Herlihy and Shavit (2012), Figure 5b). As tasks become more evenly distributed across the CPUs, the runtime ideally decreases linearly with the number of processors. As the number of CPUs increases, the workload is partitioned more efficiently, and runtime ideally scales according to $T(N) = T_1/N$, where T_1 is the runtime using one CPU and N is the product of CPUs used and the number of concurrent tasks employed. While the observed trial data may visually resemble exponential decay, it actually follows an inverse linear relationship, where the rate of runtime reduction gradually stabilizes as the number of CPUs increases. If extrapolated toward an infinite

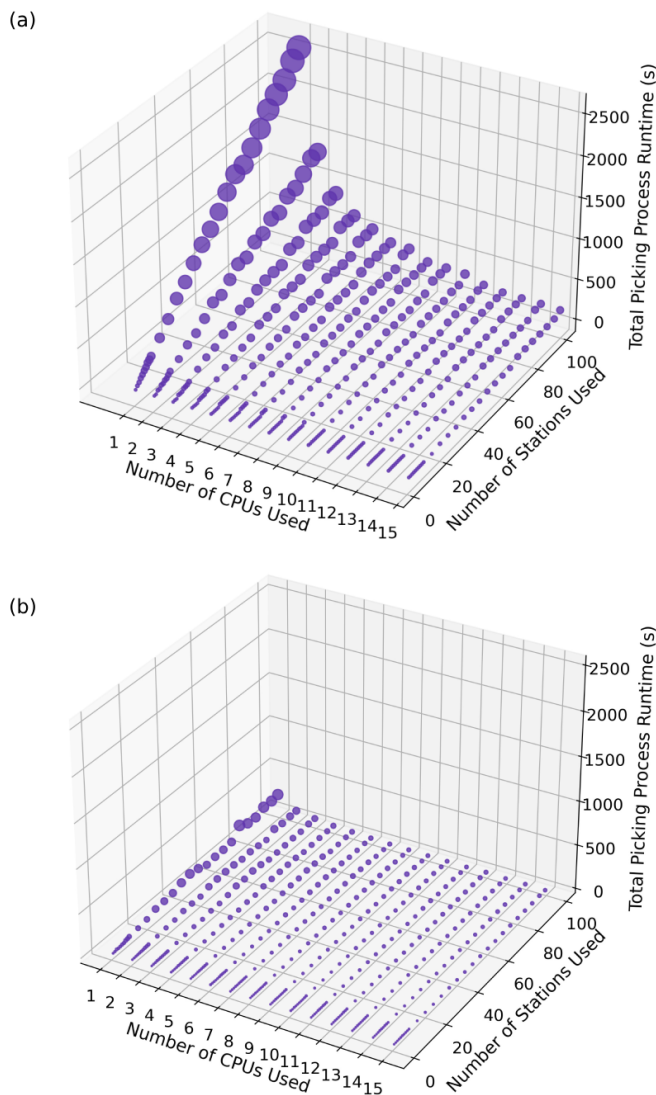


Figure 4. Simulated Real-Time Deployment Runtime Performance Comparison between EQCCTOne (a) and EQCCTPro (b): Both workflows processed the same test workload using identical computational resources. The significant reduction in runtime is attributed to EQCCTPro's ability (Figure 3) to distribute the workload across all available resources through parallel processing. Our optimized parallelization approach enables multiple waveforms to be processed simultaneously—a capability previously unattainable with the sequential workflow of EQCCTOne (Figure 1). Dot size represents runtime duration, with larger dots indicating longer runtimes and smaller dots corresponding to shorter runtimes.

number of CPUs, the runtime asymptotically approaches zero—characteristic of embarrassingly parallel systems.

The fluctuations observed along the concurrent prediction axis (y-axis) likely stem from changes in parallel efficiency regimes. In most cases, increasing the number of

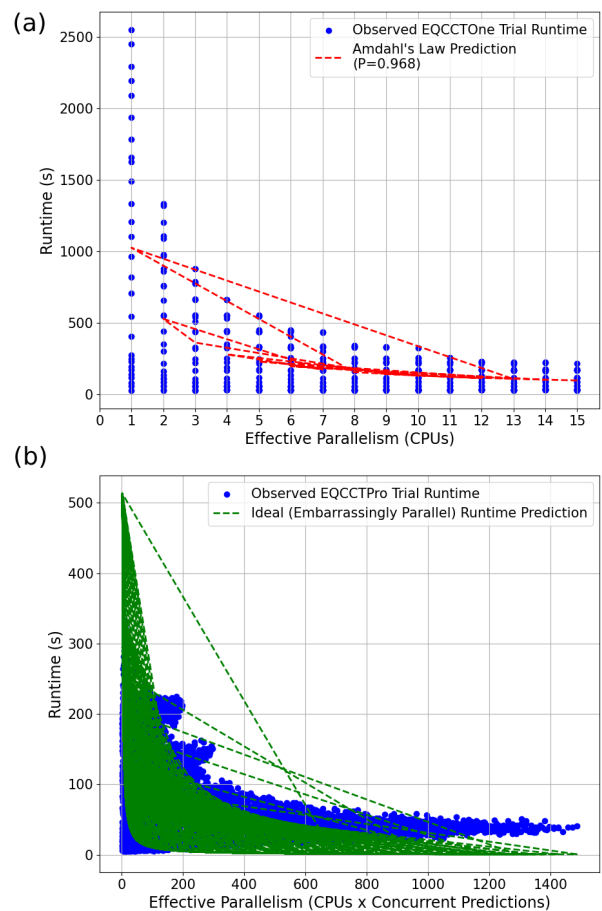


Figure 5. Modeling the Observed EQCCTOne (a) and EQCCTPro (b) Trial Data against Amdahl's Law and the Embarrassingly Parallel Model, Respectively: The observed performance of EQCCTOne (a, blue dots) closely aligns with the theoretical curve predicted by Amdahl's Law (a, dashed-red lines), reflecting diminishing returns associated with parallelizing a primarily serial workflow. On the contrary, EQCCTPro demonstrates a near-ideal inversely linear decay in runtime (b, blue dots), characteristic of an embarrassingly parallel problem (b, dashed-green lines), where independent waveform tasks can be distributed efficiently across available processors. Both implementations show strong agreement with their respective theoretical performance models.

concurrent predictions improves resource utilization and reduces idle time, especially when matched well to CPU availability. However, overloading CPUs with excessive concurrent tasks can result in imbalanced workloads, where some processors are overburdened, while others remain underutilized, leading to longer runtimes. These regime shifts manifest as jagged or non-linear behavior in runtime performance and highlight the importance of tuning con-

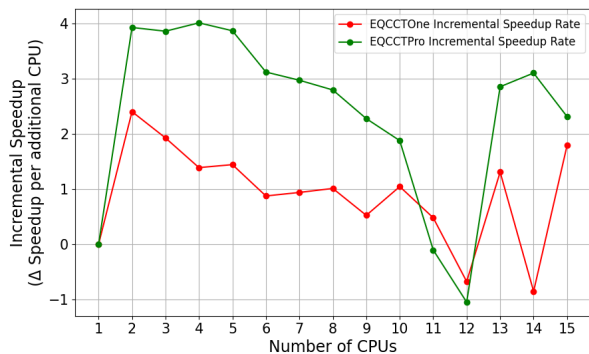


Figure 6. Incremental Speedup Comparison Between EQCCTOne and EQCCTPro: The observed speedup improvement between *EQCCTOne* (red) to *EQCCTPro* (green) clearly highlight the performance advantages of parallelization. In this context, *speedup* refers to the ratio of baseline serial runtime to parallel runtime. It indicates how much faster a task completes when distributed multiple processors (N) when compared to using fewer processors ($N - 1$). A higher speedup indicates more efficient utilization of parallel resources, resulting in improved computational efficiency. In line with Amdahl's Law (Amdahl (1967)), *EQCCTOne* demonstrates diminishing speedup returns as CPU count increases, due to its reliance on serial execution. Speedup gains plateau around 12 CPUs, and the subsequent fluctuation in incremental gains likely reflects the system overcoming certain overhead limitations—such as task scheduling or context switching. Nonetheless, the long-term gains continue to diminish as the benefits of adding additional CPUs taper off. In contrast, *EQCCTPro* exhibits consistent incremental speedup gains across lower CPU counts (1–10), reflecting its embarrassingly parallel architecture (Figure 3, Herlihy and Shavit (2012)) While a noticeable drop in speedup appears at CPUs 11–12, this is likely due to residual serial components within the software or resource contention amongst the CPUs. Nonetheless, *EQCCTPro* quickly recovers. This recovery is likely the results of the framework overcoming serial execution limitations by more effectively balancing the workload across the available processors.

currency parameters to match hardware constraints and capabilities.

The linear relationship between station counts and runtime is intuitive and expected: as the workload increases, so does the time required to process it. This reinforces the need for parallelization strategies that can scale predictably with increasing network size.

Despite these observed trends, deriving a closed-form functional relationship between these three independent variables proved challenging. Traditional linear and non-linear regression models (Johnston and DiNardo (1997); Bates and Watts (2008)) were unable to capture the highly non-linear and interactive structure of the data. Runtime

outcomes result from a complex interplay of task granularity, processor availability, concurrency strategy, and resource contention—phenomena that traditional models are ill-suited to represent.

To overcome this, **Gradient Boosting Regressor (GBR)** (Friedman (2001)), an ML technique that builds ensembles of decision trees to iteratively minimize prediction error was employed. GBR is particularly effective in high-dimensional, noisy, or non-linear datasets where feature interactions dominate. The resulting model (Figure 7b) achieved high predictive accuracy, estimating runtimes within ± 10.5 -second margin. This tool provides immediate value to seismic network operators by enabling them to estimate *EQCCTPro*'s runtime under varying workload and hardware configurations. By inputting the number of CPUs, stations, the model recommends the optimal number of concurrent predictions to use—thereby reducing the need for exhaustive empirical testing and streamlining DL deployment.

Applications and Future Work

The advancements in waveform processing enabled by *EQCCTPro*'s optimized parallelization techniques present a transformative opportunity for seismic networks worldwide. These innovations support the deployment of highly accurate, DL-based real-time seismicity monitoring across diverse observance regions. By leveraging EQCCT as a generalized DL-picker model—capable of operating effectively without region-specific fine-tuning (Saad et al. (2023), Wang et al. (2021))—*EQCCTPro* delivers high waveform processing performance while requiring only minimal, readily available computing resources. Furthermore, integrating *EQCCTPro* with the predictive capabilities of the GBR model, our methodology enables seismic networks to dynamically identify optimal station and CPU configurations for DL-based deployments. This approach maximizes efficient pick generation while ensuring flexible and effective use of available computational resources.

While these methods greatly improve the application of machine learning for real-time seismicity monitoring, further research is needed to enhance scalability and adaptability. At TexNet, ongoing efforts are focused on deploying EQCCT in real-time using the *EQCCTPro* framework, building on this research to refine deployment strategies. Efficiently analysis of short waveform time lengths from a

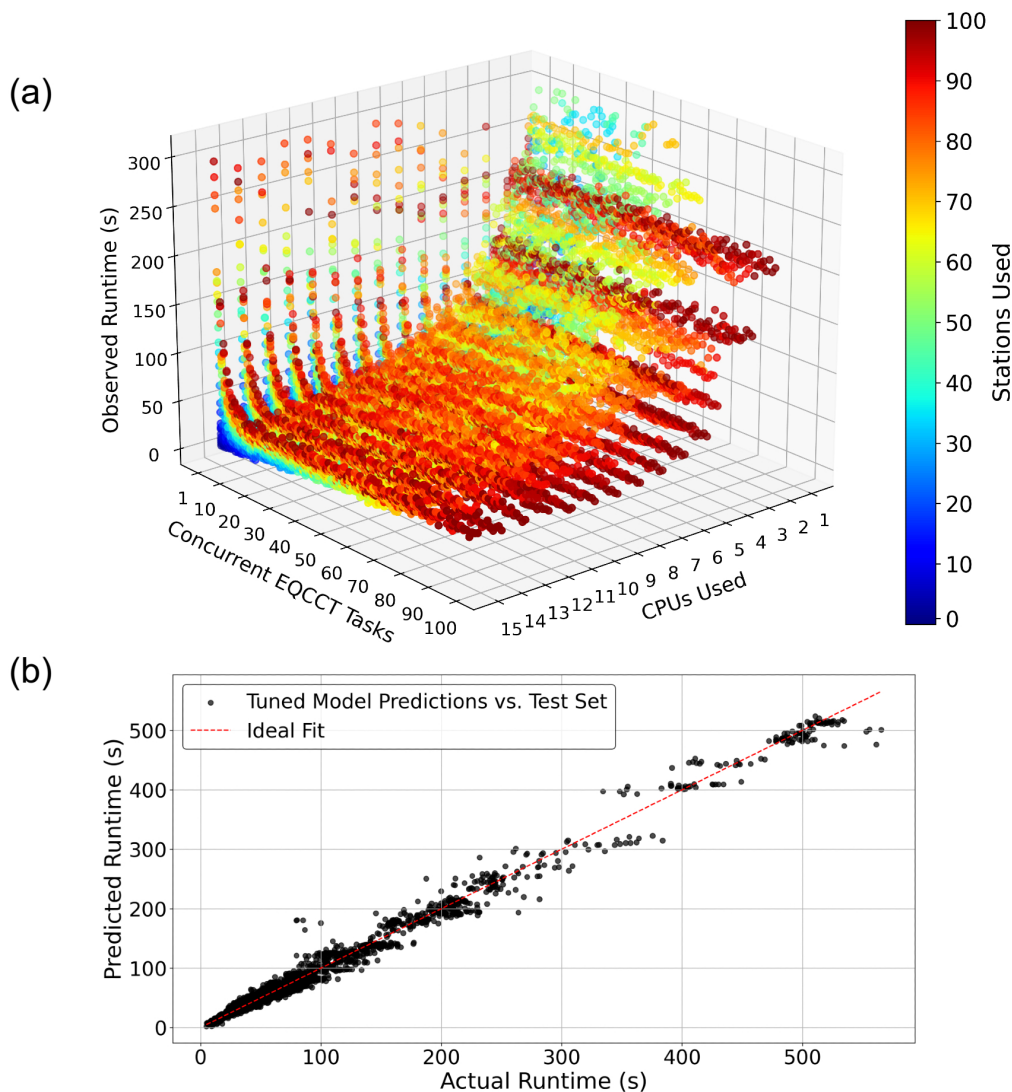


Figure 7. Analyzing and Modeling the Impact of Independent Variables—CPUs, Total Stations Processed, and Concurrent Station Predictions—on Runtime (a) and the Modeled Accuracy of the Gradient Boosting Regressor (GBR) on the Test Dataset (b): Analyzing the trial results (a), it is evident that increasing the number of CPUs is critical for reducing process runtime (**inversely linear decay curve**). As more stations are processed (**dark blue to red**), additional CPUs are required to efficiently handle the workload. Similarly, selecting an optimal number of concurrent predictions

(**y-axis**) is essential, as using too few leads to longer runtimes due to a near-sequential processing approach being applied instead of a highly parallelized one. The **GBR** (b) proved to be the most reliable model for capturing these complex relationships, achieving an **RMSE of 10.5491**. This model provides seismologists with a powerful tool to evaluate system readiness for deploying *EQCCTPro* by simulating parallelization strategies tailored to their available computational resources.

large number of stations remains a critical requirement for achieving real-time performance in operational settings.

Similarly, while this study optimized a simulated real-time deployment, future research is needed to determine the optimal configurations for *EQCCTPro* to process large wave-

form timeframes and extensive timechunk datasets that are commonly used in post-event analysis. Additionally, while the *ECCTPro* framework currently utilizes EQCCT as its DL-picker, future work should seek to incorporate other DL-

pickers, such as PhaseNet and EQTransformer (Zhu and Beroza (2018); Mousavi et al. (2020)) into the framework.

Conclusion

This study represents a significant leap in enhancing the efficiency of DL-based, seismic phase-picking detection through the development and optimization of *EQCCTPro*. By leveraging parallel processing frameworks and fine-tuning configurable parameters—such as concurrent station predictions—we overcame the performance limitations of *EQCCTOne*, achieving substantial reductions in runtime. Additionally, integrating the Gradient Boosting Regressor (GBR) into the real-time deployment workflow enables seismic networks to dynamically assess and identify optimal resource configurations for deploying *EQCCTPro*, ensuring maximum waveform processing efficiency even under constrained computational conditions.

Future research should focus on scaling these solutions to larger networks, exploring the limits of runtime performance in resource-constrained environments, and testing other DL-picking models with the *EQCCTPro* framework. Continued refinement of parallelization strategies and improvements to the GBR model—such as reducing the runtime prediction to an accuracy of one second—will further increase the system’s adaptability and usability across a wide range of seismic monitoring applications.

Data and Resources

Seismic wave data and computational hardware were provided by the Texas Seismological Network and Seismology Research Team. EQCCT is an open-source machine learning model (Saad et al. (2023)) and is available at EQCCT. *EQCCTOne* and *EQCCTPro* can be accessed at EQCCTOne and EQCCTPro Github repositories.

Declaration of Competing Interests

The authors acknowledge that there are no conflicts of interest recorded.¹

Acknowledgments

The authors would like to thank the Texas Seismological Network and Seismology Research group and the State of

Texas that provided financial support for this publication under the University of Texas at Austin award #201503664. Special thanks to Elena Kalogirou and Peter Sarkis for revising this paper.

References

- Ali, M. (2023, Jul). Distributed processing using ray framework in python.
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), New York, NY, USA, pp. 483–485. Association for Computing Machinery.
- Bates, D. and D. Watts (2008, 05). *Nonlinear Regression Analysis and Its Applications*, pp. 32 – 66.
- Chen, Y., O. M. Saad, A. Savvaids, F. Zhang, Y. Chen, D. Huang, H. Li, and F. Aziz Zanjani (2024). Deep learning for p-wave first-motion polarity determination and its application in focal mechanism inversion. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–11.
- Feng, T., S. Mohanna, and L. Meng (2022). Edgephase: A deep learning model for multi-station seismic phase picking. *Geochemistry, Geophysics, Geosystems* **23**(11), e2022GC010453. e2022GC010453 2022GC010453.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* **29**(5), 1189 – 1232.
- Gustafson, J. L. (1988, May). Reevaluating amdahl’s law. *Commun. ACM* **31**(5), 532–533.
- Herlihy, M. and N. Shavit (2012). *The Art of Multiprocessor Programming, Revised Reprint*. Morgan Kaufmann.
- Johnston, J. and J. DiNardo (1997). *Econometric Methods* (4th ed.). McGraw-Hill.
- McCool, M., J. Reinders, and A. Robison (2012). *Structured Parallel Programming: Patterns for Efficient Computation*. ITPro collection. Elsevier Science.
- Mousavi, S., W. Ellsworth, Z. Weiqiang, L. Chuang, and G. Beroza (2020, 08). Earthquake transformer—an attentive deep-learning model for simultaneous earthquake detection and phase picking. *Nature Communications* **11**, 3952.
- Mousavi, S. M. and G. C. Beroza (2022). Deep-learning seismology. *Science* **377**(6607), eabm4470.
- Münchmeyer, J., J. Woollam, A. Rietbrock, F. Tilmann, D. Lange, T. Bornstein, T. Diehl, C. Giunchi, F. Haslinger, D. Jozinović, A. Michelini, J. Saul, and H. Soto (2022, 01). Which picker fits my data? a quantitative evaluation of deep learning based seismic pickers. *Journal of Geophysical Research: Solid Earth* **127**.
- Ray.io (n.d.). What is ray core? <https://docs.ray.io/en/latest/ray-core/walkthrough.html>. Accessed: 2023-11-13.
- Saad, O. M. and Y. Chen (2022). Capsphase: Capsule neural network for seismic phase classification and picking. *IEEE Transactions on Geoscience and Remote Sensing* **60**, 1–11.

¹The authors acknowledge that there are no conflicts of interest recorded.

- Saad, O. M., Y. Chen, A. Savvaidis, W. Chen, F. Zhang, and Y. Chen (2022). Unsupervised deep learning for single-channel earthquake data denoising and its applications in event detection and fully automatic location. *IEEE Transactions on Geoscience and Remote Sensing* **60**, 1–10.
- Saad, O. M., Y. Chen, A. Savvaidis, S. Fomel, and Y. Chen (2022). Real-time earthquake detection and magnitude estimation using vision transformer. *Journal of Geophysical Research: Solid Earth* **127**(5), e2021JB023657. e2021JB023657 2021JB023657.
- Saad, O. M., Y. Chen, D. Siervo, F. Zhang, A. Savvaidis, G.-c. D. Huang, N. Igonin, S. Fomel, and Y. Chen (2023). Eqcct: A production-ready earthquake detection and phase-picking method using the compact convolutional transformer. *IEEE Transactions on Geoscience and Remote Sensing* **61**, 1–15.
- Saad, O. M., G. Huang, Y. Chen, A. Savvaidis, S. Fomel, N. Pham, and Y. Chen (2021). Scalodeep: A highly generalized deep learning framework for real-time earthquake detection. *Journal of Geophysical Research: Solid Earth* **126**(4), e2020JB021473. e2020JB021473 2020JB021473.
- Savvaidis, A., B. Young, G. D. Huang, and A. Lomax (2019, 06). Texnet: A statewide seismological network in texas. *Seismological Research Letters* **90**(4), 1702–1715.
- Tan, Y. J., F. Waldhauser, W. Ellsworth, M. Zhang, Z. Weiqiang, M. Michele, L. Chiaraluce, G. Beroza, and M. Segou (2021, 04). Machine-learning-based high-resolution earthquake catalog reveals how complex fault structures were activated during the 2016–2017 central italy sequence. *The Seismic Record* **1**, 11–19.
- Wang, T., D. Trugman, and Y. Lin (2021). Seismogen: Seismic waveform synthesis using gan with application to seismic data augmentation. *Journal of Geophysical Research: Solid Earth* **126**(4), e2020JB020077. e2020JB020077 2020JB020077.
- Xiao, Z., J. Wang, C. Liu, J. Li, L. Zhao, and Z. Yao (2021). Siamese earthquake transformer: A pair-input deep-learning model for earthquake detection and phase picking on a seismic array. *Journal of Geophysical Research: Solid Earth* **126**(5), e2020JB021444. e2020JB021444 2020JB021444.
- Yu, Z., Y. Jiang, X. Jing, and H. Zheng (2024). Study on geomagnetic observations associated with three major earthquakes in southwest china through a novel deep learning framework. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–18.
- Zhu, W. and G. C. Beroza (2018, 10). Phasenet: a deep-neural-network-based seismic arrival-time picking method. *Geophysical Journal International* **216**(1), 261–273.

Manuscript Received 00 Month 0000